INSTITUTE OF AERONAUTICAL ENGINEERING
(Autonomous)
Dundigal - 500 043, Hyderabad, Telangana

DEPARTMENT OF CSE(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

DATABASE MANAGEMENT SYSTEMS LABORATORY

Name of the Staff : Dr.B.Venkateswara Rao **Emp Id:11207**

EXERCISE –6: Triggers **Date:24/09/2026**

1. Aim:

To create a row-level trigger on the CUSTOMERS table that fires for INSERT, UPDATE, and DELETE operations and displays the difference between the old salary value and the new salary value.

---

2. Algorithm

1. Start.
2. Create a CUSTOMERS table with columns:
   - ID
   - NAME
   - AGE
   - ADDRESS
   - SALARY
3. Insert the given customer records into the table.
4. Enable DBMS_OUTPUT to display messages.
5. Create a **row-level trigger** on the CUSTOMERS table.
6. Make the trigger execute for:
   - INSERT
   - UPDATE
   - DELETE
7. For every operation:
   - For **INSERT**, consider the old salary as 0.
   - For **UPDATE**, calculate NEW SALARY - OLD SALARY.
   - For **DELETE**, consider the new salary as 0.
8. Display the old salary, new salary, and salary difference.
9. Test the trigger using INSERT, UPDATE, and DELETE.
10. Stop.

---

3. PL/SQL Source Code

Step 1: Create the CUSTOMERS table

```
CREATE TABLE CUSTOMERS (
   ID NUMBER PRIMARY KEY,
   NAME VARCHAR2(20),
   AGE NUMBER,
   ADDRESS VARCHAR2(30),
   SALARY NUMBER
);
```

Step 2: Insert the given records

```
INSERT INTO CUSTOMERS VALUES (1, 'Alive',   24, 'Khammam',  2000);
INSERT INTO CUSTOMERS VALUES (2, 'Bob',     27, 'Kadappa',  3000);
INSERT INTO CUSTOMERS VALUES (3, 'Catri',   25, 'Guntur',   4000);
INSERT INTO CUSTOMERS VALUES (4, 'Dena',    28, 'Hyderabad',5000);
INSERT INTO CUSTOMERS VALUES (5, 'Eeshwar', 27, 'Kurnool',  6000);
INSERT INTO CUSTOMERS VALUES (6, 'Farooq',  28, 'Nellur',   7000);

COMMIT;
```

Step 3: Create the Trigger

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE TRIGGER customer_salary_trigger
AFTER INSERT OR UPDATE OR DELETE
ON CUSTOMERS
FOR EACH ROW
BEGIN

   IF INSERTING THEN

      DBMS_OUTPUT.PUT_LINE('Operation : INSERT');
      DBMS_OUTPUT.PUT_LINE('New Salary : ' || :NEW.SALARY);
      DBMS_OUTPUT.PUT_LINE('Salary Difference : ' ||
                  (NVL(:NEW.SALARY, 0) - NVL(:OLD.SALARY, 0)));

   ELSIF UPDATING THEN

      DBMS_OUTPUT.PUT_LINE('Operation : UPDATE');
      DBMS_OUTPUT.PUT_LINE('Old Salary : ' || :OLD.SALARY);
      DBMS_OUTPUT.PUT_LINE('New Salary : ' || :NEW.SALARY);
      DBMS_OUTPUT.PUT_LINE('Salary Difference : ' ||
                  (NVL(:NEW.SALARY, 0) - NVL(:OLD.SALARY, 0)));

   ELSIF DELETING THEN

      DBMS_OUTPUT.PUT_LINE('Operation : DELETE');
DBMS_OUTPUT.PUT_LINE('Customer Deleted : ' || :OLD.NAME);

   END IF;

END;
/
```

4. Test the Trigger

A. Test UPDATE

Suppose we increase Bob's salary from 3000 to 4000.

```
UPDATE CUSTOMERS
SET SALARY = 4000
WHERE ID = 2;
```

Output

```
Operation : UPDATE
Old Salary : 3000
New Salary : 4000
Salary Difference : 1000
```

B. Test INSERT

```
INSERT INTO CUSTOMERS
VALUES (7, 'Ravi', 25, 'Vijayawada', 8000);
```

Output

```
Operation : INSERT
New Salary : 8000
Salary Difference : 8000
```

Here, the old salary is treated as 0.

C. Test DELETE

```
DELETE FROM CUSTOMERS
WHERE ID = 7;
```

Output

```
Operation : DELETE
Old Salary : 8000
Salary Difference : -8000
```

Here, the new salary is treated as 0.

EXERCISE – 6: TRIGGERS

2. Insert, Delete and Update Triggers using Passenger Database

Aim

To create and execute **INSERT, DELETE, and UPDATE triggers** on the PASSENGER table, including an INSERT trigger to check whether the Passport_id contains exactly six digits and triggers to display appropriate messages when records are inserted, deleted, or updated.

---

1. Create Passenger Table

SQL Structure

```
CREATE TABLE Passenger (
   Passport_id INTEGER PRIMARY KEY,
   Name VARCHAR2(50) NOT NULL,
   Age INTEGER NOT NULL,
   Sex CHAR,
   Address VARCHAR2(50) NOT NULL
);
```

---

2. Insert Sample Records

```
INSERT INTO Passenger
VALUES (123456, 'Ravi', 25, 'M', 'Hyderabad');

INSERT INTO Passenger
VALUES (234567, 'Sita', 30, 'F', 'Vijayawada');

INSERT INTO Passenger
VALUES (345678, 'Kiran', 28, 'M', 'Guntur');

INSERT INTO Passenger
VALUES (456789, 'Anitha', 32, 'F', 'Warangal');

INSERT INTO Passenger
VALUES (567890, 'Rahul', 27, 'M', 'Khammam');

COMMIT;
```

Display the records

```
SELECT * FROM Passenger;
```

Sample Output

PASSPORT_ID	NAME	AGE	SEX	ADDRESS
123456	Ravi	25	M	Hyderabad
234567	Sita	30	F	Vijayawada
345678	Kiran	28	M	Guntur

PASSPORT_ID	NAME	AGE	SEX	ADDRESS
456789	Anitha	32	F	Warangal
567890	Rahul	27	M	Khammam

A. INSERT Trigger – Check Passport ID

Aim

To create an INSERT trigger that checks whether the Passport_id contains **exactly six digits** before inserting a passenger record.

Algorithm

1. Start.
2. Create the PASSENGER table.
3. Create a BEFORE INSERT trigger.
4. Get the new Passport_id using :NEW.Passport_id.
5. Convert the ID to a character value.
6. Check whether its length is exactly 6.
7. If the length is not 6, display an error and stop the insertion.
8. If it contains exactly 6 digits, allow the insertion.
9. Stop.

Trigger

```
CREATE OR REPLACE TRIGGER check_passport_id
BEFORE INSERT ON Passenger
FOR EACH ROW
BEGIN
   IF LENGTH(TO_CHAR(:NEW.Passport_id)) != 6 THEN
      RAISE_APPLICATION_ERROR(
         -20001,
         'Passport ID must contain exactly 6 digits'
      );
   END IF;
END;
/
```

Test – Valid Passport ID

```
INSERT INTO Passenger
VALUES (678901, 'Arjun', 29, 'M', 'Nellore');
```

This record will be inserted because:

678901 → 6 digits

Test – Invalid Passport ID

```
INSERT INTO Passenger
```

```
VALUES (12345, 'Mohan', 35, 'M', 'Kurnool');
```

Output:

```
ORA-20001: Passport ID must contain exactly 6 digits
```

The record will **not** be inserted.

Note: Since Passport_id is defined as INTEGER, Oracle stores it as a number. Therefore, a value such as 012345 becomes 12345, which has only 5 digits. If passport IDs can legitimately begin with zero, VARCHAR2(6) is a better datatype.

B. Trigger to Display Insert/Delete/Update Messages

Aim

To create a row-level trigger on the PASSENGER table that displays:

```
1 Record is inserted
1 Record is deleted
1 Record is updated
```

depending on the operation performed.

Algorithm

1. Start.
2. Create a row-level trigger on PASSENGER.
3. Specify the trigger for INSERT, DELETE, and UPDATE.
4. Check whether the operation is INSERT using INSERTING.
5. If INSERT, display 1 Record is inserted.
6. Check whether the operation is DELETE using DELETING.
7. If DELETE, display 1 Record is deleted.
8. Check whether the operation is UPDATE using UPDATING.
9. If UPDATE, display 1 Record is updated.
10. Stop.

SQL Structure

First enable output:

```
SET SERVEROUTPUT ON;
```

Then create the trigger:

```
CREATE OR REPLACE TRIGGER passenger_operation_trigger
AFTER INSERT OR DELETE OR UPDATE
ON Passenger
FOR EACH ROW
```

```
BEGIN

   IF INSERTING THEN
      DBMS_OUTPUT.PUT_LINE('1 Record is inserted');

   ELSIF DELETING THEN
      DBMS_OUTPUT.PUT_LINE('1 Record is deleted');

   ELSIF UPDATING THEN
      DBMS_OUTPUT.PUT_LINE('1 Record is updated');

   END IF;

END;
/
```

---

3. Test INSERT Operation

```
INSERT INTO Passenger
VALUES (678901, 'Arjun', 29, 'M', 'Nellore');
```

Output

```
1 Record is inserted
```

---

4. Test UPDATE Operation

Change Arjun's address:

```
UPDATE Passenger
SET Address = 'Chennai'
WHERE Passport_id = 678901;
```

Output

```
1 Record is updated
```

Check the record:

```
SELECT * FROM Passenger
WHERE Passport_id = 678901;
```

---

5. Test DELETE Operation

```
DELETE FROM Passenger
WHERE Passport_id = 678901;
```

Output

```
1 Record is deleted
```

EXERCISE – 6: TRIGGERS

3. Insert Row in Employee Table Using Triggers

Aim

To create and execute **BEFORE INSERT, BEFORE UPDATE, and BEFORE DELETE triggers** on an EMPLOYEE table. The triggers will:

1. Convert the employee name to **uppercase** whenever a record is inserted or updated.
2. Before deleting an employee, store the deleted record in a DELETE_EMP table along with the **username, date, and time of deletion**.
3. Create a **transparent audit system** for CUST_MSTR to keep track of records that are updated or deleted.

A. Employee Table

1. Structure of EMPLOYEE Table

```
CREATE TABLE EMPLOYEE (
   EMP_ID NUMBER PRIMARY KEY,
   EMP_NAME VARCHAR2(50),
   AGE NUMBER,
   DEPT VARCHAR2(30),
   SALARY NUMBER(10,2)
);
```

Sample Records

```
INSERT INTO EMPLOYEE VALUES
(101, 'Ravi', 25, 'IT', 30000);

INSERT INTO EMPLOYEE VALUES
(102, 'Sita', 28, 'HR', 35000);

INSERT INTO EMPLOYEE VALUES
(103, 'Kiran', 30, 'SALES', 40000);

COMMIT;
```

Check the table:

```
SELECT * FROM EMPLOYEE;
```

B. Convert Employee Name to Uppercase

Aim

To create a trigger that automatically converts the employee name into **uppercase** whenever an employee record is inserted or updated.

Algorithm

1. Start.
2. Create the EMPLOYEE table.
3. Create a BEFORE INSERT OR UPDATE trigger.
4. Get the employee name using :NEW.EMP_NAME.
5. Convert the name to uppercase using UPPER().
6. Store the uppercase value back into :NEW.EMP_NAME.
7. Allow the INSERT or UPDATE operation.
8. Stop.

PL/SQL Trigger

```
CREATE OR REPLACE TRIGGER emp_name_uppercase
BEFORE INSERT OR UPDATE
ON EMPLOYEE
FOR EACH ROW
BEGIN
   :NEW.EMP_NAME := UPPER(:NEW.EMP_NAME);
END;
/
```

Test INSERT

```
INSERT INTO EMPLOYEE
VALUES (104, 'arjun', 27, 'IT', 32000);
```

Check:

```
SELECT * FROM EMPLOYEE
WHERE EMP_ID = 104;
```

Output

```
104   ARJUN   27   IT   32000
```

Although we inserted:

```
arjun
```

the trigger automatically changed it to:

```
ARJUN
```

Test UPDATE

```
UPDATE EMPLOYEE
```

```
SET EMP_NAME = 'rahul'
WHERE EMP_ID = 104;
```

Check:

```
SELECT * FROM EMPLOYEE
WHERE EMP_ID = 104;
```

Output:

```
104   RAHUL   27   IT   32000
```

C. Trigger Before Deleting an Employee

The question asks us to:

Before deleting a record from EMP, insert the row into DELETE_EMP and record the user who deleted it and the date/time of deletion.

1. Create DELETE_EMP Table

We need a separate table to store deleted employee records.

```
CREATE TABLE DELETE_EMP (
   EMP_ID NUMBER,
   EMP_NAME VARCHAR2(50),
   AGE NUMBER,
   DEPT VARCHAR2(30),
   SALARY NUMBER(10,2),
   DELETED_BY VARCHAR2(30),
   DELETED_DATE DATE
);
```

---

2. Algorithm

1. Start.
2. Create the DELETE_EMP table.
3. Create a BEFORE DELETE trigger on EMPLOYEE.
4. Obtain the old employee details using :OLD.
5. Insert the old employee details into DELETE_EMP.
6. Get the current database user using USER.
7. Get the current date and time using SYSDATE.
8. Store these details in DELETE_EMP.
9. Allow the original DELETE operation to continue.
10. Stop.

---

3. PL/SQL Trigger

```
CREATE OR REPLACE TRIGGER emp_delete_backup
BEFORE DELETE
ON EMPLOYEE
FOR EACH ROW
BEGIN

   INSERT INTO DELETE_EMP
   (
      EMP_ID,
      EMP_NAME,
      AGE,
      DEPT,
      SALARY,
      DELETED_BY,
      DELETED_DATE
   )
   VALUES
   (
      :OLD.EMP_ID,
      :OLD.EMP_NAME,
      :OLD.AGE,
      :OLD.DEPT,
      :OLD.SALARY,
      USER,
      SYSDATE
   );

END;
/
```

Test the Trigger

Delete employee 104:

```
DELETE FROM EMPLOYEE
WHERE EMP_ID = 104;
```

Now check the deleted records:

```
SELECT * FROM DELETE_EMP;
```

You will get something similar to:

EMP_ID	EMP_NAME	AGE	DEPT	SALARY	DELETED_BY	DELETED_DATE
104	RAHUL	27	IT	32000	SYSTEM	24-SEP-26

DELETED_BY depends on the Oracle user you are logged in as.

D. Transparent Audit System for CUST_MSTR

Aim

To create an audit system that automatically records information whenever a record in the CUST_MSTR table is **updated or deleted**.

The audit table will store:

- Customer ID
- Old customer name
- Old address
- Old phone
- Operation performed
- User who performed the operation
- Date and time

---

1. Create CUST_MSTR Table

```
CREATE TABLE CUST_MSTR (
   CUST_ID NUMBER PRIMARY KEY,
   CUST_NAME VARCHAR2(50),
   ADDRESS VARCHAR2(100),
   PHONE VARCHAR2(15)
);
```

Sample Records

```
INSERT INTO CUST_MSTR
VALUES (201, 'Ravi', 'Hyderabad', '9876543210');

INSERT INTO CUST_MSTR
VALUES (202, 'Sita', 'Vijayawada', '9876543211');

INSERT INTO CUST_MSTR
VALUES (203, 'Kiran', 'Guntur', '9876543212');

COMMIT;
```

---

2. Create Audit Table

```
CREATE TABLE CUST_MSTR_AUDIT (
   CUST_ID NUMBER,
   CUST_NAME VARCHAR2(50),
   ADDRESS VARCHAR2(100),
   PHONE VARCHAR2(15),
   OPERATION VARCHAR2(10),
   USER_NAME VARCHAR2(30),
   AUDIT_DATE DATE
);
```

---

3. Audit Trigger

Algorithm

1. Start.
2. Create the CUST_MSTR_AUDIT table.
3. Create an AFTER UPDATE OR DELETE trigger on CUST_MSTR.
4. If an UPDATE occurs:
    - Take the old values using :OLD.
    - Store them in the audit table.
    - Record UPDATE as the operation.
5. If a DELETE occurs:
    - Take the old values using :OLD.
    - Store them in the audit table.
    - Record DELETE as the operation.
6. Record the current Oracle user using USER.
7. Record the current date/time using SYSDATE.
8. Stop.

---

4. PL/SQL Trigger

```
CREATE OR REPLACE TRIGGER cust_mstr_audit
AFTER UPDATE OR DELETE
ON CUST_MSTR
FOR EACH ROW
BEGIN

  IF UPDATING THEN

    INSERT INTO CUST_MSTR_AUDIT
    (
      CUST_ID,
      CUST_NAME,
      ADDRESS,
      PHONE,
      OPERATION,
      USER_NAME,
      AUDIT_DATE
    )
    VALUES
    (
      :OLD.CUST_ID,
      :OLD.CUST_NAME,
      :OLD.ADDRESS,
      :OLD.PHONE,
      'UPDATE',
      USER,
      SYSDATE
    );

  ELSIF DELETING THEN

    INSERT INTO CUST_MSTR_AUDIT
    (
      CUST_ID,
      CUST_NAME,
```

```
        ADDRESS,
        PHONE,
        OPERATION,
        USER_NAME,
        AUDIT_DATE
    )
    VALUES
    (
        :OLD.CUST_ID,
        :OLD.CUST_NAME,
        :OLD.ADDRESS,
        :OLD.PHONE,
        'DELETE',
        USER,
        SYSDATE
    );

  END IF;

END;
/
```

---

5. Test UPDATE Audit

```
UPDATE CUST_MSTR
SET ADDRESS = 'Chennai'
WHERE CUST_ID = 201;
```

Now:

```
SELECT * FROM CUST_MSTR_AUDIT;
```

The audit table will contain something like:

CUST_ID	CUST_NAME	ADDRESS	PHONE	OPERATION	USER_NAME
201	Ravi	Hyderabad	9876543210	UPDATE	SYSTEM

Notice that the audit table contains the **old address (Hyderabad)**, even though the current customer address is Chennai.

---

6. Test DELETE Audit

```
DELETE FROM CUST_MSTR
WHERE CUST_ID = 202;
```

Then:

```
SELECT * FROM CUST_MSTR_AUDIT;
```

You will see:

CUST_ID	CUST_NAME	ADDRESS	PHONE	OPERATION	USER_NAME
201	Ravi	Hyderabad	9876543210	UPDATE	SYSTEM
202	Sita	Vijayawada	9876543211	DELETE	SYSTEM

Important Trigger Concepts

Concept	Meaning
BEFORE INSERT	Trigger executes before inserting
BEFORE UPDATE	Trigger executes before updating
BEFORE DELETE	Trigger executes before deleting
AFTER UPDATE	Trigger executes after updating
AFTER DELETE	Trigger executes after deleting
:NEW	New value of a column
:OLD	Previous value of a column
UPPER()	Converts text to uppercase
USER	Current Oracle username
SYSDATE	Current database date and time
INSERTING	Checks whether INSERT occurred
UPDATING	Checks whether UPDATE occurred
DELETING	Checks whether DELETE occurred
FOR EACH ROW	Creates a row-level trigger

Result

Thus, the required triggers were successfully created:

1. EMP_NAME_UPPERCASE — converts employee names to uppercase during INSERT/UPDATE.
2. EMP_DELETE_BACKUP — saves deleted employee records along with username and deletion date/time.
3. CUST_MSTR_AUDIT — provides a transparent audit trail for UPDATE and DELETE operations.